Nama: ALDI SAPUTRA

NIM: 105841115923

Pertemuan 04: Klasifikasi (Logistic Regression & Decision Tree)

Tujuan Praktikum: Pada praktikum pertemuan keempat ini, fokus materi bergeser dari sekadar menebak angka (regresi) menjadi mengelompokkan data ke dalam kategori tertentu, atau yang biasa disebut dengan *Classification* (Klasifikasi). Tujuan utama tugas kali ini adalah membandingkan dua algoritma klasifikasi yang sangat populer di dunia Machine Learning, yaitu *Logistic Regression* dan *Decision Tree*. Dataset yang saya jadikan bahan eksperimen adalah dataset bunga Iris, sebuah dataset klasik yang fungsinya untuk mengklasifikasikan bunga ke dalam tiga spesies yang berbeda berdasarkan ukuran kelopak dan mahkotanya.Penjelasan Langkah Kerja

Penjelasan Langkah Kerja Secara Detail

Persiapan dan Pemotongan Data (*Data Splitting*)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Langkah paling awal yang saya lakukan adalah memanggil library yang dibutuhkan dari sklearn, mulai dari datasetnya sendiri (load_iris), fungsi untuk memotong data (train_test_split), algoritma modelnya (LogisticRegression dan DecisionTreeClassifier), sampai alat ukur evaluasinya seperti accuracy_score, classification_report, dan confusion_matrix.

```
# Load dataset Iris
# ==========================================================
# Dataset Iris digunakan untuk klasifikasi multi-kelas (3 spesies).
iris = load_iris()
# Split data: 80% training dan 20% testing.
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)
```

Setelah dataset Iris di-*load*, saya tidak bisa langsung menyuapkan semua data itu ke dalam model. Data tersebut harus saya bagi dua menggunakan train_test_split. Di kode ini, saya mengatur porsinya menjadi 80% untuk *Training* (data yang dipakai model untuk belajar) dan 20% sisanya untuk *Testing* (data ujian untuk mengetes kepintaran model). Pemisahan ini sangat krusial biar ujiannya nanti *fair*; model diuji menggunakan data 20% yang belum pernah ia lihat sama sekali sebelumnya. Parameter random_state=42 juga saya tambahkan agar hasil acakan datanya selalu konsisten setiap kali saya menjalankan ulang *cell* kodenya.

Proses Pelatihan Model (*Training*)

Di tahap ini, saya langsung melatih dua model secara bersamaan agar nantinya bisa diadu performanya:

```
# Latih 2 model
# ==========================================================
# 1. Logistic Regression: Menggunakan fungsi sigmoid untuk probabilitas.
lr = LogisticRegression(max_iter=200, multi_class='multinomial').fit(X_train, y_train)

# 2. Decision Tree: Membagi data berdasarkan kriteria seperti Gini Impurity.
dt = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_train, y_train)
```

- **Model Pertama: Logistic Regression** Walaupun namanya ada kata "Regression", algoritma ini sebenarnya dipakai untuk klasifikasi dengan memanfaatkan fungsi sigmoid untuk menghitung probabilitas. Karena bunga Iris ini punya 3 kelas (bukan cuma 2 pilihan seperti benar/salah), maka saya harus menambahkan pengaturan khusus yaitu multi_class='multinomial' agar modelnya paham bahwa ini adalah klasifikasi multi-kelas. Selain itu, saya set max_iter=200 untuk memberikan waktu/batasan iterasi yang cukup bagi model agar bisa menemukan perhitungan matematis terbaiknya tanpa berhenti mendadak alias *error*. Model ini kemudian dilatih memakai perintah .fit(X_train, y_train).

- **Model Kedua: Decision Tree** Berbeda dengan model pertama yang sangat matematis, *Decision Tree* bekerja layaknya manusia yang sedang membuat keputusan bercabang berdasarkan aturan tertentu (menggunakan kriteria seperti *Gini Impurity*). Di kode ini, saya memberikan batasan max_depth=3. Ini sengaja saya lakukan agar "pohon keputusan" yang terbentuk maksimal hanya memiliki kedalaman 3 tingkat cabang. Kalau dibiarkan terlalu dalam tanpa batas, pohonnya akan terlalu rumit dan hanya menghafal data *training* (fenomena *overfitting*).

Ujian dan Evaluasi Model (Tugas 1, 2, dan 3)

Setelah kedua model selesai "belajar", sekarang saatnya menguji mereka menggunakan data 20% yang disembunyikan tadi (X_test). Saya membuat sebuah perulangan (for loop) untuk mengevaluasi LogReg dan DecisionTree secara bergantian.

```
# Tugas 1 & 2: Evaluasi kedua model
# ---------------------------------------------------------
results = {}
for name, model in {"LogReg": lr, "DecisionTree": dt}.items():
    pred = model.predict(X_test)
    acc = round(accuracy_score(y_test, pred), 3)
    results[name] = acc

    print(f"\n{'='*50}")
    print(f"{name} — Accuracy: {acc}")
    print(f"{'='*50}")

    # Tugas 3: Confusion Matrix
    # Menampilkan tabel True Positive, True Negative, False Positive, dan False Negative.
    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, pred))

    # Tugas 2: Precision, Recall, F1-Score
    # Precision: Ketepatan prediksi positif. Recall: Kemampuan mendeteksi kelas positif.
    print("\nClassification Report:")
    print(classification_report(y_test, pred, target_names=iris.target_names))
```

Untuk evaluasinya, saya menggunakan tiga metrik penilaian yang sangat detail:

1. **Accuracy Score (Skor Akurasi):** Ini adalah cara paling sederhana, yaitu menghitung berapa persen tebakan model yang benar dari total seluruh tebakan. Hasil akurasi ini saya bulatkan tiga angka di belakang koma biar lebih rapi.

2. **Confusion Matrix (Matriks Kebingungan):** Ini jauh lebih menarik dari sekadar akurasi. *Confusion Matrix* menampilkan tabel yang merincikan angka tebakan *True Positive*, *True Negative*, *False Positive*, dan *False Negative*. Lewat tabel ini, saya bisa tahu persis "di bunga spesies mana sih model ini sering keliru atau tertukar tebakannya?".

3. **Classification Report:** Matrik ini membongkar lebih dalam lagi performa model menggunakan tiga skor utama:

    - *Precision:* Mengukur ketepatan prediksi positif (kalau model nebak itu bunga jenis A, seberapa yakin tebakan itu benar?).

    - *Recall:* Mengukur kemampuan mendeteksi kelas positif (dari seluruh bunga jenis A yang ada, berapa banyak yang berhasil dikenali oleh model?).

- *F1-Score:* Merupakan nilai rata-rata harmonis gabungan antara *Precision* dan *Recall* yang sangat berguna kalau datanya tidak seimbang. Laporan ini langsung memakai target penamaan bawaan (iris.target_names) agar hasilnya berupa teks nama bunga asli dan bukan cuma angka 0, 1, 2.

Kesimpulan Akhir (Tugas 4)

```
# Tugas 4: Kesimpulan
# ============================================================
print("\n" + "=" * 50)
print("KESIMPULAN")
print("=" * 50)
best = max(results, key=results.get)
print(f"• LogReg accuracy       : {results['LogReg']}")
print(f"• DecisionTree accuracy : {results['DecisionTree']}")

if results["LogReg"] == results["DecisionTree"]:
    print(f"→ Kedua model memiliki accuracy yang sama ({results[best]}).")
    print("  Insight: Logistic Regression sangat efektif untuk dataset linear yang kecil,")
    print("  sementara Decision Tree memberikan interpretasi visual yang baik melalui struktur tree-nya.")
else:
    print(f"→ {best} memiliki accuracy lebih tinggi ({results[best]}).")

print("\n• Catatan: Decision Tree lebih rentan terhadap overfitting jika max_depth terlalu tinggi.")
print("• Catatan: Logistic Regression pada dataset ini menggunakan pendekatan multinomial untuk multi-kelas.")

print("\n✅ Pertemuan 04 selesai.")
```

Berdasarkan perbandingan kinerja kedua model yang sudah dieksekusi di *cell* terakhir, program secara otomatis mengecek mana yang terbaik. Dari pengerjaan ini, saya bisa menarik beberapa wawasan (*insight*) dan catatan penting:

1. Jika akurasi LogReg dan DecisionTree kebetulan menunjukkan angka yang sama setelah diuji, itu menandakan bahwa data Iris ini cukup bersih dan polanya mudah dipahami oleh kedua jenis algoritma.

2. **Kelebihan LogReg:** *Logistic Regression* terbukti sangat tangguh, stabil, dan efektif untuk menangani dataset yang tidak terlalu besar dan memiliki pola penyebaran data yang linier (lurus/beraturan).

3. **Kelebihan Decision Tree:** Algoritma ini ibarat "buku terbuka". Ia memberikan keunggulan di sisi interpretasi karena kita bisa secara visual melihat struktur cabang "*tree*"-nya untuk mengerti alasan di balik sebuah prediksi.

4. **Kelemahan/Catatan Evaluasi:** Saya mencatat bahwa *Decision Tree* ini sangat rawan terkena penyakit *overfitting*. Kalau nilai parameter max_depth disetel terlalu tinggi,

alih-alih mencari pola umum, ia hanya akan menghafal mati titik-titik data *training* sehingga bakal kebingungan saat diberi data baru.

Secara keseluruhan, pertemuan keempat ini sukses memberikan pemahaman mendalam tentang bagaimana mengevaluasi model klasifikasi secara komprehensif, tidak hanya bergantung pada angka akurasi semata. Pertemuan 04 selesai.